

Esercizi Sistemi Operativi

Rocco Mitola

10 giugno 2026

Indice

1	Esercizi sul disco	3
1.1	Tempo di seek di una testina	3
	Esercizio 1.1.1	4
1.2	Tempo massimo richiesto per raggiungere il settore di un hard disk	4
	Esercizio 1.2.1	5
2	Esercizi sulla schedulazione	6
2.1	Contesto teorico	6
2.2	Schedulazione FCFS	6
	Esercizio 2.2.1	6
2.3	Schedulazione SJF	9
	Esercizio 2.3.1	9
2.4	Schedulazione RR	12
	Esercizio 2.4.1	12
	Esercizio 2.4.2	14
3	Deadlock	16
3.1	Modello del Sistema e delle Risorse	16
3.2	Il Deadlock: Le Condizioni di Coffman	17
3.3	Rappresentazione Grafica: Il Grafo di Allocazione delle Risorse (RAG)	18
	3.3.1 Confronto: Quando un Ciclo causa Deadlock	18
3.4	Stati Sicuri e Sicurezza del Sistema	19
3.5	Strategie di Gestione del Deadlock	19
	Esercizio 3.5.1	20
3.6	Schema risolutivo per l'Algoritmo del Banchiere	21

Esercizio 3.6.1	23
4 Memoria Centrale	25
4.1 Contesto Teorico	25
4.2 Esercizi sull'EAT e Hit Ratio	25
Esercizio 4.2.1	25
Esercizio 4.2.2	26
4.3 Dimensionamento e Segmentazione	27
Esercizio 4.3.1	27
Esercizio 4.3.2	29
5 Memoria Virtuale: Algoritmi di Sostituzione delle Pagine	29
5.1 Contesto Teorico e Funzionamento	29
5.2 Guida Metodologica alla Lettura delle Tabelle	30
5.3 Esercizio Ripassativo Completo	31
Esercizio 5.3.1	31
6 Struttura delle Memorie di Massa e Schedulazione del Disco	34
6.1 Struttura e Indirizzamento Fisico	34
6.2 Schedulazione delle Testine	34
Esercizio 6.2.1	34
6.3 Calcolo del Tempo di Seek basato su Seek Unitario	35
6.4 Amministrazione e Gestione del Disco	35
7 Implementazione del File System	36
7.1 Architettura e Strutture Dati	36
7.2 Metodi di Allocazione	36
Esercizio 7.2.1	36
7.3 Prestazioni e Recupero	37

1 Esercizi sul disco

Questi esercizi si suddividono in due tipologie:

1. Dato il cilindro iniziale, il cilindro finale, il numero totale dei cilindri e il tempo massimo di seek, trovare il tempo di seek di una testina;
2. Data la velocità di rotazione del disco, il tempo massimo di seek, calcolare il tempo massimo richiesto per raggiungere il settore di un hard disk.

Il procedimento delle due tipologie di esercizi è standard, quindi ne faremo uno per tipologia concentrandoci su esso.

1.1 Tempo di seek di una testina

Obiettivo

L'obiettivo di questa tipologia di esercizi è cercare di capire il tempo necessario ai bracci magnetici (le testine) di un hard disk meccanico per spostarsi fisicamente sul cilindro che contiene il dato richiesto.

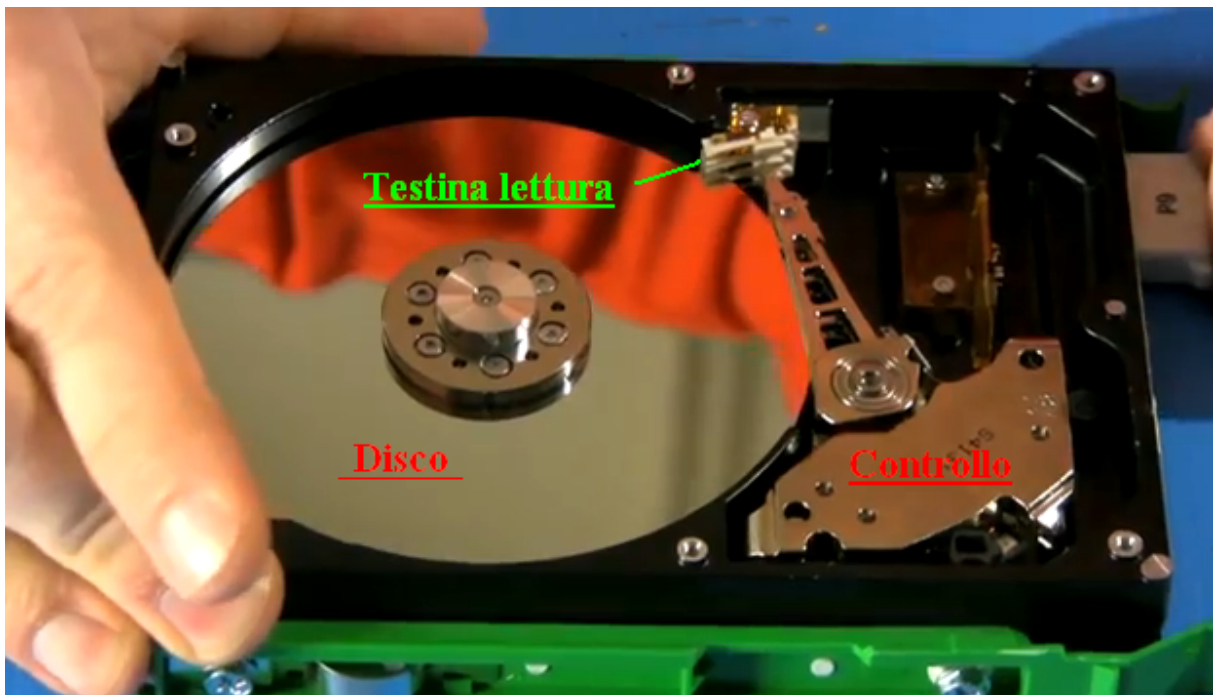


Figura 1: Hard disk

Esercizio 1.1.1

Si calcoli il tempo di seek di una testina, inizialmente posizionata sul cilindro 115, se la successiva operazione di lettura si riferisce al cilindro 195. Il tempo massimo di seek è di 35 ms ed il numero totale dei cilindri è di 1350.

Dati

- Cilindro Iniziale (C_i): 115
- Cilindro Finale (C_f): 195
- Tempo Massimo di Seek ($T_{seekMax}$): 35 ms
- Numero Totale Cilindri (C_{tot}): 1350

Svolgimento

Per prima cosa, calcoliamo il numero di cilindri da attraversare:

$$N_{cilindri} = |C_f - C_i| = |195 - 115| = 80 \text{ cilindri}$$

A questo punto procediamo con il calcolo del tempo di seek unitario:

$$T_{seekUnitario} = \frac{T_{seekMax}}{C_{tot}} = \frac{35 \text{ ms}}{1350} \approx 0.026 \frac{\text{ms}}{\text{cilindro}}$$

Il tempo di seek totale sarà dato dal prodotto del tempo di seek unitario per lo spostamento della testina:

$$T_{seek} = T_{seekUnitario} \cdot N_{cilindri} = 0.026 \cdot 80 \approx 2.074 \text{ ms}$$

1.2 Tempo massimo richiesto per raggiungere il settore di un hard disk

Obiettivo

L'obiettivo di questa tipologia di esercizi è cercare di capire il tempo massimo che le testine di un hard disk meccanico possono impiegare per raggiungere un settore qualsiasi del disco. Tale tempo è dato dalla somma del tempo massimo di seek ($T_{seekMax}$) e del tempo massimo di latenza rotazionale (T_{rotMax}).

Esercizio 1.2.1

Determinare il tempo massimo richiesto per accedere ad un settore circolare di un hard disk se il suo tempo massimo di seek è di 20 ms e il disco ruota alla velocità di 5200 RPM (giri al minuto).

Dati

- Tempo Massimo di Seek (T_{seekMax}): 20 ms
- Velocità di rotazione (V_{rot}): 5200 RPM

Svolgimento

Il tempo massimo di accesso al settore ($T_{\text{accessMax}}$) è dato dalla somma:

$$T_{\text{accessMax}} = T_{\text{seekMax}} + T_{\text{rotMax}}$$

Come primo passaggio, calcoliamo la **Velocità in RPS** (Rivoluzioni al Secondo):

$$V_{\text{RPS}} = \frac{V_{\text{rot}}}{60} = \frac{5200 \text{ giri/min}}{60 \text{ s/min}} \approx 86.67 \frac{\text{giri}}{\text{s}}$$

A questo punto calcoliamo il **Tempo di Latenza Rotazionale Massima** (T_{rotMax}), che è il tempo di una rotazione completa (Periodo):

$$T_{\text{rotMax}} = \frac{1}{V_{\text{RPS}}} = \frac{1}{86.67 \frac{\text{giri}}{\text{s}}} \approx 0.01154 \text{ s}$$

Convertiamo in millisecondi:

$$T_{\text{rotMax}} = 0.01154 \text{ s} \cdot 1000 \frac{\text{ms}}{\text{s}} \approx 11.54 \text{ ms}$$

Infine, calcoliamo il tempo di accesso massimo:

$$T_{\text{accessMax}} = T_{\text{seekMax}} + T_{\text{rotMax}} = 20 \text{ ms} + 11.54 \text{ ms} = 31.54 \text{ ms}$$

2 Esercizi sulla schedulazione

2.1 Contesto teorico

La schedulazione dei processi è il componente del sistema operativo che si occupa di decidere quali dei processi pronti per l'esecuzione devono essere assegnati alla CPU e in quale ordine.

Esistono diversi algoritmi di schedulazione, che possono favorire una categoria di processi piuttosto che un'altra.

In generale esistono dei criteri di comparazione tra i processi:

- Utilizzo della CPU - mantenere la CPU il più impegnata possibile;
- Frequenza di completamento (**throughput**) - numero massimo di processi completati per unità di tempo;
- Tempo di completamento - intervallo che va dal momento dell'immissione del processo nel sistema al momento del completamento;
- Tempo di attesa - somma dei tempi spesi in attesa nella coda dei processi pronti;
- Tempo di risposta - tempo che intercorre dalla formulazione della richiesta fino alla produzione della prima risposta.

2.2 Schedulazione FCFS

Il First-Come, First-Served, "primo arrivato, primo servito" (FCFS) è un algoritmo di schedulazione che esegue i processi nell'ordine in cui arrivano, non tenendo conto di altre variabili. È un algoritmo di tipo non-preemptive, questo significa che la CPU non interrompe l'esecuzione del processo.

Esercizio 2.2.1

Calcolare il tempo di attesa medio di questi tre processi quando arrivano nei seguenti ordini:

- P_1, P_2, P_3 ;

- P_2, P_3, P_1

Definizione

Un picco di CPU (o CPU Burst) è l'intervallo di tempo continuativo in cui un processo ha bisogno di utilizzare il processore per eseguire calcoli, prima di fermarsi per attendere un evento (come un'operazione di Input/Output) o prima di essere interrotto dal sistema. I processi alternano fasi in cui usano la CPU e fasi in cui aspettano dati da disco/rete/tastiera

Nella tabella seguente sono esplicitate le durate dei picchi di CPU necessari per ogni processo.

Processo	Durata Picco (ms)
P_1	24
P_2	3
P_3	3

Formula

Il tempo medio di attesa si calcola come il quoziente della somma dei tempi di attesa dei processi singoli ed il numero di processi:

$$\text{Attesa media} = \frac{\sum_{i=0}^n P_i}{N_{\text{processi}}}$$

Svolgimento

Analizziamo i due casi richiesti disegnando il **Diagramma di Gantt**, che mostra l'assegnazione della CPU nel tempo.

Caso 1: Ordine P_1, P_2, P_3

- P_1 arriva al tempo 0 e occupa la CPU per 24 ms (da 0 a 24).
- P_2 attende che P_1 finisca, poi esegue per 3 ms (da 24 a 27).
- P_3 attende che P_1 e P_2 finiscano, poi esegue per 3 ms (da 27 a 30).



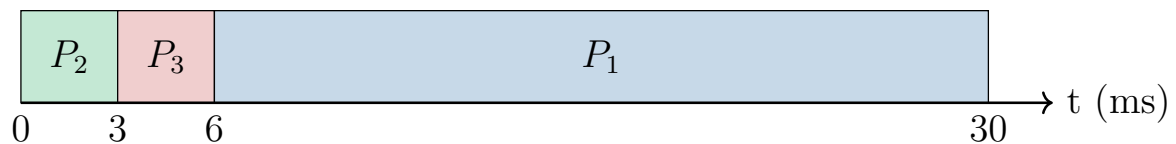
Calcolo dei tempi di attesa:

- P_1 : attende 0 ms.
- P_2 : attende 24 ms.
- P_3 : attende 27 ms.

$$\text{Attesa media} = \frac{0 + 24 + 27}{3} = \frac{51}{3} = 17 \text{ ms}$$

Caso 2: Ordine P_2, P_3, P_1 Se l'ordine di arrivo cambia, cambia drasticamente l'efficienza (convoy effect):

- P_2 esegue subito per 3 ms (da 0 a 3).
- P_3 esegue subito dopo per 3 ms (da 3 a 6).
- P_1 esegue per ultimo per 24 ms (da 6 a 30).



Calcolo dei tempi di attesa:

- P_2 : attende 0 ms.
- P_3 : attende 3 ms.
- P_1 : attende 6 ms.

$$\text{Attesa media} = \frac{0 + 3 + 6}{3} = \frac{9}{3} = 3 \text{ ms}$$

Osservazione: Notiamo come l'ordine di arrivo influenzi pesantemente il tempo medio di attesa (da 17ms a 3ms), dimostrando la debolezza dell'algoritmo FCFS con processi di durata molto diversa.

2.3 Schedulazione SJF

La schedulazione SJF (Shortest Job First) è un algoritmo di gestione dei processi che assegna la CPU al processo con il tempo di esecuzione (burst time) più breve tra quelli in coda. Esistono due versioni:

- **Non-preemptive** – quando un processo arriva nella coda dei processi pronti mentre un altro processo è ancora in esecuzione, l'algoritmo attende che il processo corrente termini il suo picco di CPU (non lo interrompe);
- **Preemptive** – quando un processo arriva nella coda dei processi pronti, l'algoritmo confronta la sua durata con il tempo rimanente del processo in esecuzione. Se il nuovo processo è più breve, quello corrente viene interrotto (preemption) per cedere la CPU. Questa variante è nota come Shortest-Remaining-Time-First (SRTF).

L'algoritmo nel caso preemptive tiene conto sia del tempo di arrivo che della durata del picco. In generale, un processo con un picco di CPU minore sarà eseguito prima di uno con durata maggiore. Di seguito verrà mostrato un esercizio di esempio:

Esercizio 2.3.1

Calcolare il tempo medio di attesa di questi processi utilizzando la schedulazione SJF non-preemptive e la SJF preemptive:

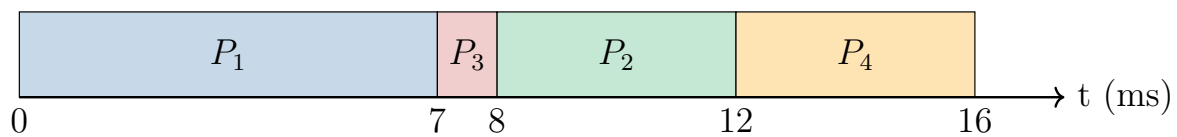
Processo	Arrivo (ms)	Durata (ms)
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

Analizziamo i due casi richiesti disegnando il **Diagramma di Gantt**, che mostra l'assegnazione della CPU nel tempo.

Caso 1: SJF non-preemptive

- P_1 arriva al tempo 0 e occupa la CPU per 7 ms (da 0 a 7).

- al tempo 7 sono arrivati P_2 , P_3 e P_4 con durate di picco rispettivamente di 4ms, 1ms e 4ms. Prendiamo il processo con durata minore di picco, nel nostro caso P_3 , che arriva al tempo 4 e occupa la CPU per 1ms (da 7 a 8).
- A questo punto ci rimangono P_2 e P_4 che hanno lo stesso tempo di picco (4ms), quindi diamo priorità al processo P_2 , poiché è arrivato prima. Eseguiamo quindi P_2 che è arrivato al tempo 2 con durata di picco di 4ms (da 8 a 12).
- Eseguiamo per ultimo P_4 per 4 ms (da 12 a 16).



Nel calcolo dei tempi di attesa dobbiamo considerare anche il tempo di arrivo del processo

- P_1 attende 0 ms.
- P_3 viene eseguito al tempo 7 ed arriva al tempo 4, quindi attende $7 - 4 = 3$ ms.
- P_2 viene eseguito al tempo 8 ed arriva al tempo 2, quindi attende $8 - 2 = 6$ ms.
- P_4 viene eseguito al tempo 12 ed arriva al tempo 5, quindi attende $12 - 5 = 7$ ms

$$\text{Attesa media} = \frac{0 + 3 + 6 + 7}{4} = \frac{16}{4} = 4 \text{ ms}$$

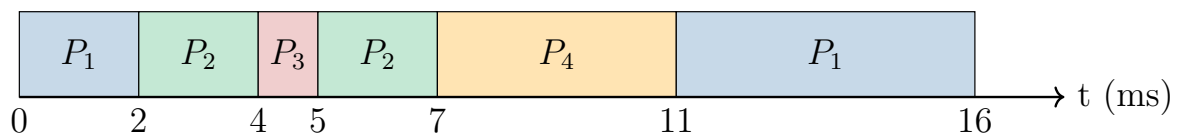
Caso 2: SJF preemptive In questo caso se durante l'esecuzione di un processo arriva un processo con tempo di picco minore, l'esecuzione del processo viene arrestata per eseguire il processo con tempo di picco minore

- Al tempo 0 l'unico processo in coda è P_1 , quindi iniziamo la sua esecuzione;
- Al tempo 2 arriva in coda il processo P_2 con durata di picco 4 ms, che è minore della durata residua del processo P_1 . Quindi stoppiamo P_1 ed eseguiamo P_2 ;
- Al tempo 4 arriva P_3 con durata di picco di 1 ms, quindi stoppiamo l'esecuzione di P_2 ed eseguiamo P_3 ;
- Al tempo 5 arriva anche P_4 con durata di picco 4. A questo punto bisogna decidere quale processo eseguire. Riassumiamo la situazione:

- Il processo P_1 è stato eseguito per 2 ms, quindi dobbiamo eseguirlo per altri 5ms;
- Il processo P_2 è stato eseguito per 2 ms, quindi dobbiamo eseguirlo per altri 2 ms;
- Il processo P_3 ha terminato la sua esecuzione;
- Il processo P_4 non è stato eseguito ed ha durata di picco 4 ms ed è arrivato al tempo 5.

La decisione dell'algoritmo di schedulazione si baserà sulla durata del tempo di picco dei processi da eseguire, qualora i processi avranno stessa durata di picco avrà la precedenza il processo che è arrivato prima. Nel nostro caso P_2 è il processo con durata del picco minore (ricorda che è stato eseguito già per 2 ms, quindi ha bisogno di altri 2 ms), quindi eseguiamo P_2 per 2ms. A questo punto dobbiamo decidere chi eseguire tra P_4 e P_1 : il processo P_4 ha bisogno della CPU per 4 ms, mentre il processo P_1 ha bisogno della CPU ancora per 5ms, quindi l'ordine di esecuzione sarà P_4 e poi P_1 .

- Eseguiamo il processo P_4 per 4 ms;
- Eseguiamo il processo P_1 per i suoi restanti 5 ms



Adesso calcoliamo i tempi di attesa di P_1 , P_2 , P_3 e P_4 :

- Il processo P_1 arriva al tempo 0, viene eseguito all'istante 0 per 2 ms e poi all'istante 11 per 5 ms, quindi il suo tempo di attesa sarà dato da $0 + (11 - 2) = 9$ ms;
- Il processo P_2 arriva al tempo 2, viene eseguito al tempo 2 per 2 ms, all'istante 4 viene stoppato per 1 ms e viene rieseguito per 2 ms (dal tempo 5 al tempo 7). Quindi il suo tempo di attesa sarà $0 + (5 - 4) = 1$ ms;
- Il processo P_3 arriva al tempo 4 e viene eseguito al tempo 4 e non viene stoppato, quindi attende 0 ms;
- Il processo P_4 arriva al tempo 5 e viene eseguito al tempo 7 senza essere stoppato, quindi attende $(7 - 5) = 2$ ms

$$\text{Attesa media} = \frac{9 + 1 + 0 + 2}{4} = \frac{12}{4} = 3 \text{ ms}$$

2.4 Schedulazione RR

Si tratta di un algoritmo di tipo FCFS con l'aggiunta della preemption per migliorare l'alternanza dei processi. Ad ogni processo è assegnato un time slice (10-100 millisecondi generalmente). Se entro questo arco di tempo il processo non lascia la CPU verrà interrotto e reintrodotta, tramite interrupt nella coda dei processi pronti della CPU. Molto importante è la scelta del time slice:

- se troppo grande non ci sarà la preemption;
- se troppo piccolo, parallelizzo l'esecuzione ma aumento il context switching. Il Dispatching andrà ad aumentare il tempo in cui la CPU non è attiva per permettere di cambiare il contesto. Troppi cambi di contesto causano un rallentamento.

Nei calcolatori interattivi è ottimo perché inizia a dare subito risposte. A questo punto procediamo con un esempio per capire meglio il funzionamento del RR:

Esercizio 2.4.1

Disegniamo il diagramma di Gantt dell'esecuzione di questi processi che arrivano tutti al tempo zero, mediante l'algoritmo di schedulazione Round Robin con time slice 20 ms

Processo	Durata (ms)
P_1	53
P_2	17
P_3	68
P_4	24

- Al tempo 0 tutti i processi sono nella coda, quindi eseguo P_1, P_2, P_3, P_4 .
- A questo punto c'è da capire chi eseguo prima, quindi guardando il Gantt possiamo capire cosa è successo:
 - P_1 è stato eseguito per 20 ms ed è tornato in coda con 33 ms rimanenti;
 - P_2 con durata di tempo di poco 17 ms ha avuto bisogno di un solo time slice per terminare la sua esecuzione;

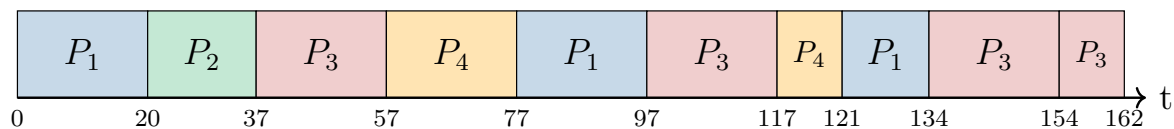
- P_3 è stato eseguito per 20 ms e quindi gli rimangono 48 ms;
- P_4 è stato eseguito per 20 ms e quindi gli rimangono 4 ms;

Eseguiamo i processi nell'ordine in cui vanno in coda. Guardando dal tempo 0 al tempo 77 in coda sono andati P_1 , P_3 e P_4 , che eseguiamo proprio nell'ordine di lettura. A questo punto la situazione è la seguente:

- P_1 è stato eseguito per altri 20 ms ed è stato messo in coda con 13 ms di tempo residuo;
- P_3 è stato eseguito per altri 20 ms ed è stato messo in coda con 28 ms di tempo residuo;
- P_4 invece ha terminato la sua esecuzione.

Eseguiamo sempre i processi nell'ordine in cui sono stati messi in coda e quindi eseguiamo di nuovo P_1 e P_3 .

A questo punto P_1 avrà terminato la sua esecuzione e quindi diamo modo a P_3 di terminare dandogli un altro time slice. Di seguito il disegno del diagramma di Gantt:



(Nota: Il diagramma di Gantt è rappresentato in scala schematica per favorire la leggibilità dei blocchi più brevi)

Procediamo con il calcolo dei tempi di attesa dando per assodato che partiamo sempre al tempo 0 nell'esempio:

- P_1 viene eseguito subito dal tempo 0 al tempo 20, poi al tempo 20 viene stoppato e viene rieseguito dal tempo 77 al tempo 97, successivamente viene stoppato ancora e viene rieseguito dal tempo 121 al tempo 134. Quindi $T_{a-p1} = 0 + (77 - 20) + (121 - 97) = 81$ ms;
- P_2 viene eseguito dal tempo 20 al tempo 37 quindi $T_{a-p2} = (20 - 0) = 20$ ms
- P_3 viene eseguito dal tempo 37 al tempo 57, poi al tempo 57 viene stoppato e viene rieseguito dal tempo 97 al tempo 117, successivamente viene stoppato e viene rieseguito al tempo 134. Quindi $T_{a-p3} = (37 - 0) + (97 - 57) + (134 - 117) = 94$ ms
- P_4 viene eseguito dal tempo 57 al tempo 77, poi al tempo 77 viene stoppato e viene rieseguito dal tempo 117 al tempo 121. Quindi $T_{a-p4} = (57 - 0) + (117 - 77) = 97$ ms

Il tempo di attesa medio sarà:

$$\text{Attesa media} = \frac{81 + 20 + 94 + 97}{4} = \frac{292}{4} = 73 \text{ ms}$$

A questo punto può sorgere spontanea una domanda, cosa succede se i processi non arrivano tutti nello stesso momento? Se durante l'esecuzione di P_1 ad esempio fosse arrivato P_2 ? Analizziamo insieme questa tematica con il prossimo esercizio

Esercizio 2.4.2

Disegniamo il diagramma di Gantt dell'esecuzione di questi processi mediante l'algoritmo di schedulazione Round Robin con time slice 2 ms

Processo	Arrivo (ms)	Durata (ms)
P_1	0	6
P_2	3	8
P_3	7	6
P_4	1	12

Svolgimento

Questo esercizio richiede molta attenzione poiché dobbiamo gestire la coda dei processi pronti (*Ready Queue*) dinamicamente, tenendo conto sia dei tempi di arrivo sia delle scadenze dei *time slice*. Si consiglia di seguire la cronologia passo-passo osservando il diagramma di Gantt riportato di seguito.

- **Intervallo $[0, 2]$ ms:** All'istante 0 l'unico processo presente è P_1 , che prende il controllo della CPU. Al tempo 1 arriva P_4 e si posiziona in coda. Al tempo 2 il *time slice* di P_1 scade; il processo viene interrotto e si risistema in coda dietro P_4 .
 - **Coda a $t = 2$:** $\langle P_4, P_1 \rangle$
 - **Tempi residui:** $P_1 : 4 \text{ ms}$, $P_4 : 12 \text{ ms}$.
- **Intervallo $[2, 4]$ ms:** Viene eseguito P_4 . Durante la sua esecuzione, all'istante 3, arriva il processo P_2 che si accoda in fondo. Al tempo 4 scade il *time slice* di P_4 , che si rimette in coda.

- **Coda a $t = 4$:** $\langle P_1, P_2, P_4 \rangle$
- **Tempi residui:** $P_1 : 4$ ms, $P_2 : 8$ ms, $P_4 : 10$ ms.
- **Intervallo $[4, 6]$ ms:** Tocca nuovamente a P_1 essere eseguito per 2 ms. In questo intervallo non arrivano nuovi processi. Al tempo 6 scade il suo turno e torna in fondo alla coda.
 - **Coda a $t = 6$:** $\langle P_2, P_4, P_1 \rangle$
 - **Tempi residui:** $P_1 : 2$ ms, $P_2 : 8$ ms, $P_4 : 10$ ms.
- **Intervallo $[6, 12]$ ms:**
 - **Da 6 a 8 ms:** Viene eseguito P_2 . Al tempo 7 arriva P_3 e va in coda. Al tempo 8, P_2 viene interrotto e torna in fondo. Coda: $\langle P_4, P_1, P_3, P_2 \rangle$. (Residuo $P_2 : 6$ ms).
 - **Da 8 a 10 ms:** Viene eseguito P_4 . Al tempo 10 scade il *time slice* e torna in fondo. Coda: $\langle P_1, P_3, P_2, P_4 \rangle$. (Residuo $P_4 : 8$ ms).
 - **Da 10 a 12 ms:** Viene eseguito P_1 . Poiché il suo tempo residuo era rimasto esattamente di 2 ms, P_1 **termina** la sua esecuzione all'istante 12 senza rimettersi in coda.

Al termine di questa fase (istante 12), la situazione aggiornata è la seguente:

- **Coda a $t = 12$:** $\langle P_3, P_2, P_4 \rangle$
- **Tempi residui:** $P_1 : 0$ ms (Terminato), $P_2 : 6$ ms, $P_3 : 6$ ms, $P_4 : 8$ ms.
- **Intervallo $[12, 18]$ ms:** Eseguiamo i tre processi rimasti nell'ordine di coda, assegnando a ciascuno un *time slice* completo di 2 ms:
 - P_3 esegue tra 12 e 14 ms, poi torna in coda.
 - P_2 esegue tra 14 e 16 ms, poi torna in coda.
 - P_4 esegue tra 16 e 18 ms, poi torna in coda.

I tempi rimanenti diventano: $P_3 : 4$ ms, $P_2 : 4$ ms, $P_4 : 6$ ms.

- **Intervallo $[18, 24]$ ms:** Eseguiamo un ulteriore giro completo mantenendo lo stesso ordine di rotazione:
 - P_3 esegue tra 18 e 20 ms, poi torna in coda.

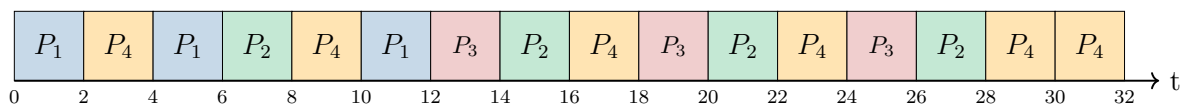
- P_2 esegue tra 20 e 22 ms, poi torna in coda.
- P_4 esegue tra 22 e 24 ms, poi torna in coda.

I tempi rimanenti scendono a: P_3 : 2 ms, P_2 : 2 ms, P_4 : 4 ms.

• **Intervallo [24, 32] ms (Fase finale):**

- **Da 24 a 26 ms:** Esegue P_3 , consuma i suoi ultimi 2 ms e **termina**.
- **Da 26 a 28 ms:** Esegue P_2 , consuma i suoi ultimi 2 ms e **termina**.
- **Da 28 a 32 ms:** Rimane solo P_4 con 4 ms residui. Esegue per un primo *time slice* (28-30) e, non essendoci altri processi pronti a contendergli il processore, continua l'esecuzione per gli ultimi 2 ms (30-32), **terminando** definitivamente il calcolo.

Di seguito viene disegnato il Gantt:



Procediamo ora con il calcolo dell'attesa media:

- T_{P_1} : $0 + (4 - 2) + (10 - 6) = 6$ ms;
- T_{P_2} : $(6 - 3) + (14 - 8) + (20 - 16) + (26 - 22) = 17$ ms;
- T_{P_3} : $(12 - 7) + (18 - 14) + (24 - 20) = 13$ ms;
- T_{P_4} : $(2 - 1) + (8 - 4) + (16 - 10) + (22 - 18) + (28 - 24) = 19$ ms

Il tempo medio di attesa sarà:

$$\text{Attesa Media} = \frac{6+17+13+19}{4} = 13.75 \text{ ms}$$

3 Deadlock

3.1 Modello del Sistema e delle Risorse

In un sistema informatico, i processi necessitano di risorse per progredire nella loro esecuzione. Queste risorse possono essere di natura fisica (come i cicli di CPU, lo spa-

zio in memoria RAM o le periferiche di I/O) o di natura logica (come file, semafori o monitor).

Il sistema è composto da diversi tipi di risorse $R_1, R_2, \dots, R_m$, e ciascun tipo può disporre di una o più **istanze** identiche (esemplari). Se un processo richiede una risorsa, l'assegnazione di un qualunque esemplare di quel tipo soddisferà la richiesta.

Il ciclo di vita dell'utilizzo di una risorsa da parte di un processo si articola in tre fasi sequenziali:

1. **Richiesta:** Il processo richiede la risorsa al sistema operativo. Se non è immediatamente disponibile, il processo viene sospeso e accodato in attesa del rilascio da parte di altri.
2. **Uso:** Una volta ottenuta la risorsa, il processo può operare su di essa (es. scrivere su un nastro magnetico o accedere a una zona di memoria).
3. **Rilascio:** Al termine delle operazioni, il processo libera la risorsa, rendendola nuovamente disponibile per il sistema.

Definizione di Deadlock

Si ha una condizione di stallo (deadlock) quando ogni processo appartenente a un insieme è in attesa di un evento (il rilascio di una risorsa) che può essere generato solo da un altro processo appartenente allo stesso insieme. In questo stato, il progresso dei processi è interrotto a tempo indefinito.

3.2 Il Deadlock: Le Condizioni di Coffman

Un deadlock non si verifica in modo casuale, ma è il risultato della coesistenza simultanea di quattro condizioni necessarie. Se riusciamo a impedirne anche solo una, lo stallo non può verificarsi.

- **Mutua Esclusione:** Almeno una risorsa deve essere utilizzata in modalità non condivisibile; ovvero, solo un processo alla volta può impegnare una specifica istanza.
- **Possesso e Attesa (Hold and Wait):** Un processo deve già detenere almeno una risorsa e, contemporaneamente, trovarsi in attesa di acquisirne altre che sono attualmente occupate da altri processi.
- **Assenza di Preemption (No Preemption):** Le risorse non possono essere sottratte forzatamente a un processo. Solo il processo che la detiene può liberarla volontariamente dopo aver completato il proprio compito.

- **Attesa Circolare:** Esiste un insieme di processi $\{P_0, P_1, \dots, P_n\}$ tale che P_0 attende una risorsa detenuta da P_1 , P_1 attende P_2 , ..., e infine P_n attende una risorsa detenuta da P_0 .

3.3 Rappresentazione Grafica: Il Grafo di Allocazione delle Risorsse (RAG)

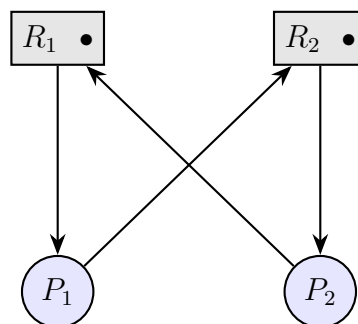
Per visualizzare e diagnosticare lo stato del sistema, utilizziamo i **Grafi di Allocazione delle Risorsse**. Questo strumento grafico ci permette di identificare immediatamente le dipendenze tra processi e risorse.

- **Nodi Processo (P_i):** Rappresentati come cerchi.
- **Nodi Risorsa (R_j):** Rappresentati come rettangoli. I punti all'interno del rettangolo indicano il numero di esemplari disponibili per quel tipo di risorsa.
- **Arco di Richiesta ($P_i \rightarrow R_j$):** Un arco orientato che indica che il processo è in attesa di una risorsa.
- **Arco di Assegnazione ($R_j \rightarrow P_i$):** Un arco orientato che indica che un'istanza della risorsa è stata allocata al processo.

3.3.1 Confronto: Quando un Ciclo causa Deadlock

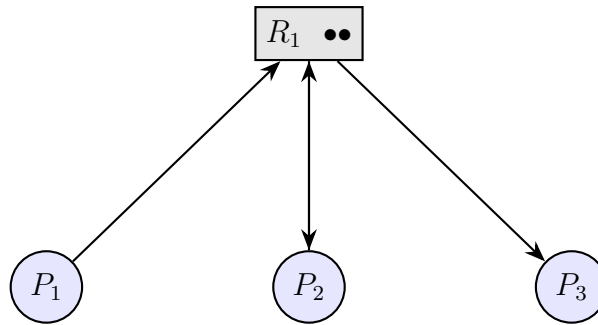
La semplice presenza di un ciclo nel grafo non sempre indica uno stallo. La distinzione fondamentale dipende dal numero di istanze:

A. Deadlock Certo (Istanze Singole) Se ogni tipo di risorsa ha un'unica istanza, la presenza di un ciclo implica necessariamente un deadlock. Nessun processo nel ciclo potrà mai ricevere la risorsa mancante.



CICLO CHIUSO = STALLO

B. Possibile Deadlock (Istanze Multiple) Se le risorse hanno più istanze, un ciclo indica solo una *possibilità* di stallo. Se un processo esterno al ciclo (che detiene una delle risorse coinvolte) termina, il ciclo si rompe.



NESSUN DEADLOCK (P3 può sbloccare il sistema)

3.4 Stati Sicuri e Sicurezza del Sistema

Un concetto chiave per evitare i deadlock è lo studio degli **stati sicuri**. Il sistema si trova in uno stato sicuro se può allocare risorse a ogni processo (fino al suo massimo dichiarato) in un ordine tale da permettere a tutti di terminare senza mai bloccarsi.

Sequenza Sicura

Una sequenza di processi $\langle P_1, P_2, \dots, P_n \rangle$ è definita **sicura** se, per ogni P_i , le risorse che esso può ancora richiedere possono essere soddisfatte dalle risorse attualmente disponibili sommate a quelle già possedute dai processi precedenti (P_j con $j < i$). Se questa condizione è verificata, P_i può attendere che i predecessori finiscano, ottenere ciò che serve, completare il compito e rilasciare tutto.

- **Stato Sicuro:** Esiste almeno una sequenza sicura. Il sistema garantisce l'assenza di stalli.
- **Stato Non Sicuro:** Non esiste alcuna sequenza sicura. Questo non significa che il deadlock sia già presente, ma che il sistema non può più impedire che una sfortunata sequenza di richieste porti allo stallo.
- **Evitamento (Avoidance):** L'obiettivo è monitorare ogni richiesta e concederla solo se il sistema rimane in uno stato sicuro.

3.5 Strategie di Gestione del Deadlock

Il sistema operativo può gestire lo stallo seguendo quattro filosofie principali:

1. Prevenzione (Deadlock Prevention):

Si agisce in anticipo impedendo strutturalmente che una delle quattro condizioni di Coffman si verifichi:

- **Negare Mutua Esclusione:** Difficile, poiché alcune risorse (es. stampanti) sono intrinsecamente non condivisibili.
 - **Negare Possesso e Attesa:** Obbligare i processi a richiedere tutte le risorse prima dell'avvio o a richiederle solo quando non ne hanno altre. Comporta un inefficiente utilizzo delle risorse.
 - **Imporre Preemption:** Se un processo richiede una risorsa occupata, gli vengono sottratte forzatamente le risorse che già detiene finché non può ottenerle tutte insieme.
 - **Negare Attesa Circolare:** Imporre un ordinamento numerico globale alle risorse. I processi devono richiedere le risorse solo in ordine crescente.
2. Evitamento (Deadlock Avoidance) Il sistema richiede ai processi informazioni aggiuntive (il **massimo bisogno**) e utilizza algoritmi predittivi per valutare lo stato di sicurezza.

Algoritmo del Banchiere: È l'algoritmo più noto per istanze multiple. Quando arriva una richiesta, il sistema simula l'allocazione: se lo stato risultante è sicuro, la risorsa viene concessa; altrimenti, il processo viene sospeso anche se la risorsa è fisicamente libera.

Esercizio 3.5.1

Consideriamo un sistema con risorse $A(10), B(5), C(7)$ e 5 processi. Dati i vettori di *Allocation* e *Max*, calcolare la matrice **Need** ($Need = Max - Allocation$) e verificare se il sistema è in uno stato sicuro con la sequenza $\langle P_2, P_4, P_5, P_1, P_3 \rangle$.

Processo	Allocation (ABC)	Max (ABC)	Available (ABC)
P_1	010	753	332
P_2	200	322	
P_3	302	902	
P_4	211	222	
P_5	002	433	

Svolgimento

Calcoliamo il bisogno residuo (Need) per ogni processo:

- $P_1 : 753 - 010 = \mathbf{743}$
- $P_2 : 322 - 200 = \mathbf{122}$
- $P_3 : 902 - 302 = \mathbf{600}$
- $P_4 : 222 - 211 = \mathbf{011}$
- $P_5 : 433 - 002 = \mathbf{431}$

Verifichiamo la sequenza:

- (a) **P2:** Chiede (1, 2, 2), Available è (3, 3, 2). OK. Termina e rilascia: $Available = (3, 3, 2) + (2, 0, 0) = (\mathbf{5}, \mathbf{3}, \mathbf{2})$.
- (b) **P4:** Chiede (0, 1, 1), Available è (5, 3, 2). OK. Termina e rilascia: $Available = (5, 3, 2) + (2, 1, 1) = (\mathbf{7}, \mathbf{4}, \mathbf{3})$.
- (c) ... procedendo così per tutti i processi, si dimostra che la sequenza è sicura.

3. Rilevamento e Ripristino (Detection and Recovery) Si permette al sistema di entrare in deadlock, ma si esegue periodicamente un algoritmo di rilevazione (es. ricerca di cicli nel *wait-for graph*). Una volta rilevato lo stallo, si interviene tramite:

- **Terminazione:** Terminare tutti i processi coinvolti o uno alla volta finché il ciclo non si rompe.
- **Prelazione:** Sottrarre risorse a un processo (vittima), effettuare un **rollback** a uno stato precedente e monitorare per evitare la **starvation** (ovvero che lo stesso processo sia sempre scelto come vittima).

4. Ignoranza (The Ostrich Algorithm) Molti sistemi operativi moderni (come UNIX e Windows) ignorano del tutto il problema, assumendo che gli stalli siano rari e che il costo del monitoraggio costante superi i benefici. In caso di blocco, l'utente dovrà riavviare il sistema o terminare manualmente i processi.

3.6 Schema risolutivo per l'Algoritmo del Banchiere

La risoluzione degli esercizi sullo stato di sicurezza richiede l'applicazione di un protocollo rigoroso suddiviso in tre fasi:

1. **Verifica di ammissibilità e coerenza delle risorse:**

- **Validità dell'allocazione:** Per ogni processo P_i , deve risultare $Allocation_i \leq Max_i$. Se un processo detiene più risorse del suo massimo dichiarato, lo stato è considerato invalido.
 - **Calcolo della matrice dei bisogni:** Si determina il bisogno residuo tramite la formula $Need = Max - Allocation$. Ogni elemento della matrice $Need$ deve essere non negativo per garantire la consistenza dello stato.
 - **Consistenza del sistema:** Si calcola il vettore delle risorse totali $Tot = \sum_{i=1}^n Allocation_i + Available_i$. Nessuna riga della matrice Max può essere più grande di Tot , poiché la richiesta massima non può superare la disponibilità totale del sistema .
2. **Algoritmo di Sicurezza (Ricerca della Sequenza):** L'algoritmo simula l'esecuzione dei processi per stabilire se il sistema si trova in uno stato sicuro.
- **Inizializzazione:** Si definiscono i vettori di lavoro $Work = Available$ e di stato $Finish[i] = false$ per ogni processo $i = 1, \dots, n$.
 - **Procedura iterativa:**
 - (a) Si individua un processo P_i non ancora terminato ($Finish[P_i] = false$) il cui bisogno può essere soddisfatto dalle risorse correnti ($Need_i \leq Work$)
 - (b) Qualora individuato, si simula il completamento del processo e il rilascio delle sue risorse: $Work = Work + Allocation_i$ e si imposta $Finish[P_i] = true$.
 - (c) Si ripete la ricerca tra i processi rimanenti fino a esaurimento dei candidati.
3. **Verifica dei risultati e conservazione:**
- **Stato Sicuro:** Il sistema è in uno stato sicuro se tutti i processi hanno completato la simulazione ($Finish = true$ per tutti).
 - **Stato Insicuro:** Se uno o più processi restano bloccati ($Finish = false$), lo stato è definito non sicuro, indicando un rischio concreto di deadlock.
 - **Verifica di conservazione delle risorse:** Al termine della sequenza sicura, il vettore $Work$ finale deve coincidere esattamente con il vettore delle risorse totali Tot calcolato nella fase iniziale.

Adesso facciamo un esempio:

Esercizio 3.6.1

Si consideri un sistema che si trova nello stato descritto dalla seguente tabella:

Processo	Allocation (ABCD)	Max (ABCD)	Available (ABCD)
P_1	0022	0023	1531
P_2	0045	3457	
P_3	2111	3762	
P_4	4445	6655	
P_5	1643	1743	

Si analizzi l'ammissibilità dello stato e si verifichi, tramite l'algoritmo del banchiere, se il sistema è in uno stato sicuro, esplicitando l'evoluzione del vettore *Work*.

Svolgimento

Per determinare se il sistema è in grado di soddisfare le richieste future, dobbiamo innanzitutto calcolare quanto ogni processo potrebbe ancora richiedere al sistema operativo. Questo dato è rappresentato dalla matrice dei bisogni residui ($Need = Max - Allocation$):

- $P_1: (0, 0, 2, 3) - (0, 0, 2, 2) = \mathbf{0001}$
- $P_2: (3, 4, 5, 7) - (0, 0, 4, 5) = \mathbf{3412}$
- $P_3: (3, 7, 6, 2) - (2, 1, 1, 1) = \mathbf{1651}$
- $P_4: (6, 6, 5, 5) - (4, 4, 4, 5) = \mathbf{2210}$
- $P_5: (1, 7, 4, 3) - (1, 6, 4, 3) = \mathbf{0100}$

Quindi la matrice *Need* è la seguente:

Processo	Need (ABCD)
P_1	0001
P_2	3412
P_3	1651
P_4	2210
P_5	0100

Prima di iniziare la simulazione, verifichiamo la consistenza delle risorse. Sommiamo tutte le istanze attualmente allocate ai processi con quelle disponibili nel vettore *Available*. Otteniamo il totale delle risorse del sistema: $Tot = [8, 16, 18, 17]$. Poiché nessun processo ha dichiarato un bisogno massimo superiore a questo totale, lo stato è considerato **ammissibile**.

Procediamo ora con la simulazione, utilizzando un vettore di lavoro *Work* inizialmente uguale alle risorse disponibili: $Work = [1, 5, 3, 1]$.

- Analizzando i bisogni, notiamo che P_1 richiede solo un'unità della risorsa D. Avendo a disposizione $[1, 5, 3, 1]$, la sua richiesta è soddisfacibile. Immaginiamo quindi che P_1 termini il suo compito e restituisca le risorse detenute. Il nostro budget sale a: $[1, 5, 3, 1] + [0, 0, 2, 2] = [1, 5, 5, 3]$.
- Con le nuove disponibilità, osserviamo P_5 . Il suo bisogno è di una sola unità di B. Avendone 5, possiamo servirlo. Al termine, P_5 rilascia un quantitativo importante di risorse, portando il budget a: $[1, 5, 5, 3] + [1, 6, 4, 3] = [2, 11, 9, 6]$.
- Scelgo P_3 , che richiede $(1, 6, 5, 1)$. Poiché $[1, 6, 5, 1] \leq [2, 11, 9, 6]$, P_3 può completare l'esecuzione. Una volta rilasciate le risorse, il vettore di lavoro diventa: $[2, 11, 9, 6] + [2, 1, 1, 1] = [4, 12, 10, 7]$.
- Proseguiamo con P_2 . Il suo bisogno $(3, 4, 1, 2)$ è ora inferiore alle risorse correnti $[4, 12, 10, 7]$. Dopo il suo rilascio, otteniamo: $[4, 12, 10, 7] + [0, 0, 4, 5] = [4, 12, 14, 12]$.
- Infine, rimane P_4 . Il suo bisogno $(2, 2, 1, 0)$ è ampiamente coperto. Concludendo la simulazione, il vettore finale sarà: $[4, 12, 14, 12] + [4, 4, 4, 5] = [8, 16, 18, 17]$

Nota bene che se all'inizio avessimo scelto P_5 sarebbe stato giusto lo stesso.

Conclusione: Siamo riusciti a completare tutti i processi. Il sistema si trova quindi in uno **stato sicuro**. La sequenza di completamento individuata che garantisce l'assenza di stallo è: $\langle P_1, P_5, P_3, P_2, P_4 \rangle$. Possiamo notare che il vettore *Work* finale coincide perfettamente con il totale delle risorse calcolato all'inizio, a conferma della correttezza della simulazione.

4 Memoria Centrale

4.1 Contesto Teorico

Parametri di Prestazione

Per il calcolo del tempo effettivo di accesso alla memoria in sistemi dotati di **TLB** (Translation Lookaside Buffer), si utilizzano i seguenti parametri:

- ϵ (**Associative Lookup**): Tempo necessario per la ricerca di una voce all'interno del TLB.
- β (**Ciclo di memoria**): Tempo richiesto per un singolo accesso alla memoria principale (RAM).
- α (**Hit ratio**): Probabilità di trovare la traduzione dell'indirizzo nel TLB.

La formula per il tempo di accesso effettivo (**EAT**) è definita come media pesata tra il caso di successo (hit) e quello di insuccesso (miss) nel TLB:

$$EAT = \alpha(\beta + \epsilon) + (1 - \alpha)(2\beta + \epsilon) = \beta(2 - \alpha) + \epsilon \quad (1)$$

Nota: Nel caso di miss ($1 - \alpha$), il sistema impiega 2β poiché deve accedere alla memoria due volte: la prima per consultare la Page Table e la seconda per prelevare il dato reale.

4.2 Esercizi sull'EAT e Hit Ratio

Esercizio 4.2.1

Si consideri un sistema con memoria paginata e TLB. Supponendo che la Page Table sia mantenuta in memoria principale, che il tempo di accesso alla memoria sia $\beta = 100$ ns e il tempo di accesso al TLB sia $\epsilon = 4$ ns con un hit ratio $\alpha = 94\%$. Calcolare l'EAT e determinare quale hit ratio servirebbe per contenere il degrado delle prestazioni entro il 10% rispetto a un accesso ideale.

Dati

- Tempo memoria (β): 100 ns
- Tempo TLB (ϵ): 4 ns

- Hit ratio (α): 0.94
- Degrado massimo: 10% (rispetto a EAT)

Svolgimento

Applichiamo la formula dell'EAT:

$$EAT = 100\text{ns} \cdot (2 - 0.94) + 4\text{ns} = 100 \cdot 1.06 + 4 = 110 \text{ ns}$$

Per calcolare l'hit ratio necessario per un degrado del 10%, poniamo il limite superiore:

$$EAT' = 1.1 * EAT = 1.1 * 110\text{ns} = 121\text{ns}$$

In questo caso specifico, l'hit ratio del 94% soddisfa esattamente il requisito del 10% di degrado. L'Hit ratio sarà:

$$\alpha' = \frac{2\beta + \epsilon - EAT'}{\beta} = 0.83$$

Esercizio 4.2.2

Si ipotizzi di poter tollerare un EAT massimo, in presenza di TLB, pari a 320 ns. Se i tempi di accesso al TLB e in memoria sono rispettivamente pari a $\epsilon = 60$ ns e $\beta = 240$ ns, si dica quale deve essere l'hit ratio (α) minimo accettabile. Come cambia l'hit ratio a seguito di un degrado delle prestazioni del 5%?

Dati

- Tempo massimo di accesso (EAT_{max}): 320 ns
- Tempo di ricerca TLB (ϵ): 60 ns
- Ciclo di memoria (β): 240 ns
- Percentuale di degrado prestazionale: 5%

Svolgimento

Utilizziamo la formula inversa per calcolare direttamente l'hit ratio minimo:

$$\alpha = \frac{2\beta + \epsilon - EAT}{\beta}$$

Sostituendo i valori:

$$\alpha = \frac{2(240) + 60 - 320}{240} = \frac{480 + 60 - 320}{240} = \frac{220}{240} \approx \mathbf{0.9167}$$

In caso di un **degrado delle prestazioni del 5%**, il nuovo tempo limite diventa:

$$EAT_{new} = 320 \cdot 1.05 = 336 \text{ ns}$$

Ricalcoliamo l'hit ratio con la medesima formula:

$$\alpha_{new} = \frac{2(240) + 60 - 336}{240} = \frac{540 - 336}{240} = \frac{204}{240} = \mathbf{0.85}$$

Suggerimento Metodologico

In sede d'esame, qualora non si ricordasse la formula inversa per l'hit ratio (α), è possibile ricavarla impostando la disequazione basata sulla definizione fondamentale di EAT:

$$EAT \leq EAT_{max}$$

$$(\beta + \epsilon)\alpha + (2\beta + \epsilon)(1 - \alpha) \leq EAT_{max}$$

Risolvendo la disequazione rispetto ad α , si otterrà lo stesso valore ottenuto dalla formula inversa

4.3 Dimensionamento e Segmentazione

Esercizio 4.3.1

Calcolare il numero di bit necessari per indirizzare una memoria virtuale paginata di 64 GB suddivisa in frame da 16 kB. Ipotizzando che ogni entry della Page Table occupi 4 B, si determini la dimensione totale della tabella in memoria.

Dati

- Memoria Virtuale (MV): $64 \text{ GB} = 2^{36} \text{ Byte}$
- Dimensione Frame/Pagina (S): $16 \text{ kB} = 2^{14} \text{ Byte}$
- Dimensione Entry (PTE): $4 \text{ B} = 2^2 \text{ Byte}$

Svolgimento

Iniziamo determinando la struttura dell'indirizzo logico. Per coprire uno spazio di indirizzamento di 64 GB, calcoliamo l'esponente in base 2:

$$64 \text{ GB} = 2^6 \cdot 2^{30} \text{ B} = 2^{36} \text{ B}$$

Pertanto, sono necessari **36 bit** per l'indirizzamento logico totale.

La dimensione della pagina è 16 kB, che corrisponde a:

$$16 \text{ kB} = 2^4 \cdot 2^{10} \text{ B} = 2^{14} \text{ B}$$

Questo valore indica che sono necessari **14 bit** per rappresentare l'offset all'interno della pagina (d). Di conseguenza, i bit destinati al numero di pagina (p) sono:

$$p = \text{bit totali} - d = 36 - 14 = 22 \text{ bit}$$

Procediamo ora al calcolo del numero di entry nella Page Table, che coincide con il numero di pagine virtuali:

$$N_{\text{pagine}} = \frac{\text{Memoria Totale}}{\text{Dimensione Pagina}} = \frac{2^{36}}{2^{14}} = 2^{22} \text{ pagine}$$

Infine, determiniamo la dimensione totale della Page Table moltiplicando il numero di entry per la dimensione di ciascuna di esse:

$$\text{Dimensione}_{\text{PT}} = N_{\text{pagine}} \cdot \text{PTE} = 2^{22} \cdot 2^2 \text{ B} = 2^{24} \text{ B}$$

Convertendo il risultato in unità più leggibili:

$$2^{24} \text{ B} = \frac{2^{24}}{2^{20}} \text{ MB} = 2^4 \text{ MB} = \mathbf{16 \text{ MB}}$$

Esercizio 4.3.2

Si consideri la Segment Table seguente e si determinino gli indirizzi fisici per gli indirizzi logici: (0, 290), (1, 177), (2, 261), (3, 560).

Segmento	Base	Length (Limite)
0	577	360
1	3210	175
2	1250	264
3	2477	356

Svolgimento

Un indirizzo fisico è valido se $offset < Length$.

- (0, 290): $290 < 360$ (Vero). Indirizzo fisico: $577 + 290 = 867$.
- (1, 177): $177 < 175$ (Falso). **Trap: segmentation fault.**
- (2, 261): $261 < 264$ (Vero). Indirizzo fisico: $1250 + 261 = 1511$.
- (3, 560): $560 < 356$ (Falso). **Trap: segmentation fault.**

5 Memoria Virtuale: Algoritmi di Sostituzione delle Pagine

5.1 Contesto Teorico e Funzionamento

Nella gestione della memoria virtuale basata su paginazione su richiesta (*demand paging*), il sistema operativo alloca un certo numero di frame fisici in RAM per ciascun processo. Quando viene richiesta una pagina non presente in memoria centrale, l'hardware solleva un'interruzione di **Page Fault**. Per fare spazio alla nuova pagina, se tutti i frame allocati sono occupati, è necessario scegliere una pagina residente (detta *pagina vittima*) da rimuovere.

Bit di Modifica (Modify / Dirty Bit)

Per ottimizzare la procedura di sostituzione, la tabella delle pagine integra il **bit di modifica**. Se la pagina vittima non è stata modificata durante la sua permanenza in memoria (bit a 0), essa può essere semplicemente sovrascritta, poiché una copia identica è già presente sul disco. Se invece è stata modificata (bit a 1), il sistema operativo deve obbligatoriamente effettuare uno *swap-out* (riscrittura su disco), aumentando i tempi di I/O.

Obiettivo Prestazionale

L'obiettivo fondamentale di qualunque algoritmo di sostituzione è minimizzare il tasso di fallimento dei riferimenti alla memoria, puntando ad avere il **più basso tasso possibile di mancanza di pagine** (*page fault rate*) su una determinata sequenza di accessi, denominata **sequenza di riferimento** (*reference string*).

5.2 Guida Metodologica alla Lettura delle Tabelle

Per risolvere correttamente gli esercizi d'esame, le tabelle a griglia devono essere interpretate come un film cronologico fotogramma per fotogramma, analizzando i vettori secondo tre direttrici:

- **Lettura in Orizzontale (Le Righe):** Mostra l'evoluzione temporale di un singolo slot fisico (*Frame*). Permette di osservare per quanti passaggi una pagina riesce a risiedere nello stesso slot prima di essere sfrattata.
- **Lettura in Verticale (Le Colonne):** Rappresenta l'esatta "fotografia" istantanea della memoria RAM in quel preciso millisecondo. Leggendo una colonna dall'alto verso il basso si conosce l'insieme esatto delle pagine residenti.
- **La Regola della Colonna Precedente:** Prima di elaborare una richiesta al tempo t , è obbligatorio ispezionare la colonna verticale al tempo $t - 1$:
 - **Caso HIT:** Se la pagina richiesta è già presente nella colonna precedente, la nuova colonna viene copiata identica, senza generare Page Fault (-).
 - **Caso FAULT:** Se la pagina richiesta NON è presente nella colonna precedente, si genera un Page Fault (**PF**) e si applica l'algoritmo per scegliere la pagina vittima da sostituire.

5.3 Esercizio Ripassativo Completo

Esercizio 5.3.1

Si consideri un processo la cui sequenza di richieste di pagine sia la seguente:

$$\text{Reference String} = \langle 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 \rangle$$

Si analizzi l'andamento delle allocazioni e si determini il numero complessivo di Page Fault riscontrati applicando i seguenti schemi:

1. Algoritmo **FIFO** con 3 frame di memoria a disposizione.
2. Algoritmo **FIFO** con 4 frame di memoria a disposizione.
3. Algoritmo **Ottimo (OPT)** con 4 frame di memoria a disposizione.
4. Algoritmo **LRU (Least Recently Used)** con 4 frame di memoria a disposizione.

Dati

- Sequenza di riferimento: $\langle 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 \rangle$
- Capacità di memoria alternative: 3 frame e 4 frame fisici.

Svolgimento

Rappresentiamo l'evoluzione temporale dello stato dei frame per ciascun algoritmo. Il simbolo **PF** indicherà l'occorrenza di un Page Fault.

1. Algoritmo FIFO (3 Frame) L'algoritmo First-In, First-Out gestisce i frame come una coda pura: la prima pagina caricata in assoluto è la prima a essere scelta come vittima in caso di sostituzione, ignorando la frequenza d'uso recente.

Stringa:	1	2	3	4	1	2	5	1	2	3	4	5
Frame 1:	1	1	1	4	4	4	5	5	5	5	5	5
Frame 2:		2	2	2	1	1	1	1	1	3	3	3
Frame 3:			3	3	3	2	2	2	2	2	4	4
Stato:	PF	PF	PF	PF	PF	PF	PF	–	–	PF	PF	–

Contando le occorrenze delle allocazioni fallite, si ottengono complessivamente **9 Page Fault**.

Analisi del punto critico (passo 4): Alla richiesta della pagina 4, la colonna precedente contiene {1, 2, 3}. La pagina più anziana è la 1 (inserita al passo 1), che viene sostituita nel Frame 1.

2. Algoritmo FIFO (4 Frame) Proviamo ad aumentare la risorsa fisica portando i frame a 4 per esaminare l'andamento del medesimo algoritmo.

Stringa:	1	2	3	4	1	2	5	1	2	3	4	5
Frame 1:	1	1	1	1	1	1	5	5	5	5	4	4
Frame 2:		2	2	2	2	2	2	1	1	1	1	5
Frame 3:			3	3	3	3	3	3	2	2	2	2
Frame 4:				4	4	4	4	4	4	3	3	3
Stato:	PF	PF	PF	PF	–	–	PF	PF	PF	PF	PF	PF

In questa configurazione si registrano ben **10 Page Fault**.

Anomalia di Belady

L'algoritmo FIFO risente dell'**Anomalia di Belady**: contrariamente all'intuizione ingegneristica, per alcune specifiche sequenze di riferimento, aumentando il numero di frame fisici allocati al processo, il tasso di Page Fault **aumenta** anziché ridursi o stabilizzarsi (in questo caso specifico si passa da 9 a 10).

3. Algoritmo Ottimo (OPT / 4 Frame) Rappresenta lo schema ideale teorico non implementabile. Sostituisce la pagina residente che non verrà utilizzata per il più lungo periodo di tempo futuro.

Stringa:	1	2	3	4	1	2	5	1	2	3	4	5
Frame 1:	1	1	1	1	1	1	1	1	1	1	4	4
Frame 2:		2	2	2	2	2	2	2	2	2	2	2
Frame 3:			3	3	3	3	3	3	3	3	3	3
Frame 4:				4	4	4	5	5	5	5	5	5
Stato:	PF	PF	PF	PF	–	–	PF	–	–	–	PF	–

Spiegazione empirica delle scelte:

- All'arrivo della pagina 5 (passo 7), la colonna precedente contiene {1, 2, 3, 4}. Guardando in avanti nella stringa verso destra, l'ordine di riutilizzo futuro è $1 \rightarrow 2 \rightarrow$

3 $\rightarrow$ 4. Poiché la pagina 4 verrà usata per ultima nel futuro, viene scelta come vittima.

- All'arrivo della pagina 4 (passo 11), la colonna precedente contiene {1, 2, 3, 5}. Guardando in avanti, solo il 5 verrà riutilizzato, mentre 1, 2, 3 non ricompaiono più. Scegliamo di rimpiazzare convenzionalmente il Frame 1 (pagina 1).

Il totale registrato è pari al minimo assoluto possibile: **6 Page Fault**.

4. Algoritmo LRU (Least Recently Used / 4 Frame) Questo approccio, largamente impiegato nei sistemi reali, approssima l'algoritmo ottimo guardando all'indietro: viene scartata la pagina che non viene referenziata da più tempo.

Stringa:	1	2	3	4	1	2	5	1	2	3	4	5
Frame 1:	1	1	1	1	1	1	1	1	1	1	1	5
Frame 2:		2	2	2	2	2	2	2	2	2	2	2
Frame 3:			3	3	3	3	5	5	5	5	4	4
Frame 4:				4	4	4	4	4	4	3	3	3
Stato:	PF	PF	PF	PF	-	-	PF	-	-	PF	PF	PF

Regola Operativa di Scansione all'Indietro per LRU

Quando si verifica un Page Fault, per localizzare la vittima senza commettere errori:

1. Si isola l'insieme dei numeri presenti nella colonna verticale precedente.
2. Ci si posiziona sulla richiesta attuale e si scorre la stringa dei passaggi verso **sinistra** (andando a ritroso nel passato, se necessario anche fino all'inizio della tabella).
3. Si depennano i numeri della RAM man mano che vengono incontrati.
4. L'**ultimo numero superstite** (quello posizionato più a sinistra nella cronologia) è la pagina meno recentemente usata e va sostituita.

Analisi dettagliata della cronologia dei riferimenti nel nostro esercizio:

- **Richiesta pagina 5 (passo 7):** RAM precedente = {1, 2, 3, 4}. Scansione a ritroso dalla stringa: incontriamo 2, poi 1, poi 4, e infine 3. La pagina 3 è la meno recente ed entra il 5 nel Frame 3. RAM = {1, 2, 5, 4}.
- **Richiesta pagina 3 (passo 10):** RAM precedente = {1, 2, 5, 4}. Scansione a ritroso dalla stringa: incontriamo 2, poi 1, poi 5, e infine 4 (passo 4). Sostituiamo il 4 nel Frame 4 con la pagina 3. RAM = {1, 2, 5, 3}.

- **Richiesta pagina 4 (passo 11):** RAM precedente = {1, 2, 5, 3}. Scansione a ritroso dalla stringa: incontriamo 3, poi 2, poi 1, e infine 5 (passo 7). Il 5 è il meno recente e viene sostituito dal 4 nel Frame 3. RAM = {1, 2, 4, 3}.
- **Richiesta pagina 5 (passo 12):** RAM precedente = {1, 2, 4, 3}. Scansione a ritroso dalla stringa: incontriamo 4, poi 3, poi 2, e infine il 1 (che non veniva toccato dal passo 5). La pagina 1 è la vittima ed entra il 5 nel Frame 1. RAM = {5, 2, 4, 3}.

Il bilancio finale si attesta su **8 Page Fault**.

6 Struttura delle Memorie di Massa e Schedulazione del Disco

6.1 Struttura e Indirizzamento Fisico

I dischi rigidi vengono indirizzati come array monodimensionali di blocchi logici. Il **blocco logico** è l'unità minima di trasferimento. Il sistema mappa questi blocchi in un indirizzo fisico tridimensionale (cilindro, traccia, settore).

Componenti del Tempo di Accesso

Il tempo di accesso totale è la somma di:

- **Seek Time:** Tempo di spostamento delle testine sul cilindro corretto.
- **Latenza di rotazione:** Tempo di attesa affinché il settore desiderato ruoti sotto la testina.

6.2 Schedulazione delle Testine

L'obiettivo è minimizzare il tempo di ricerca. Consideriamo una coda di richieste: $\langle 98, 183, 37, 122, 14, 124, 65, 67 \rangle$ con testina in posizione **53**.

Esercizio 6.2.1

Calcolare il movimento totale delle testine per i principali algoritmi di schedulazione.

Svolgimento

1. FCFS (First-Come, First-Served) Percorso: $53 \rightarrow 98 \rightarrow 183 \rightarrow 37 \rightarrow 122 \rightarrow 14 \rightarrow 124 \rightarrow 65 \rightarrow 67$ Movimento: $|98 - 53| + |183 - 98| + |37 - 183| + |122 - 37| + |14 - 122| + |124 - 14| + |65 - 124| + |67 - 65| = \mathbf{640}$ cil.

2. SSTF (Shortest Seek Time First) Percorso: $53 \rightarrow 65 \rightarrow 67 \rightarrow 37 \rightarrow 14 \rightarrow 98 \rightarrow 122 \rightarrow 124 \rightarrow 183$ Movimento: $12 + 2 + 30 + 23 + 84 + 24 + 2 + 59 = \mathbf{236}$ cil.

3. SCAN (Algoritmo dell'Ascensore) Percorso: $53 \rightarrow 37 \rightarrow 14 \rightarrow 0 \rightarrow 65 \rightarrow 67 \rightarrow 98 \rightarrow 122 \rightarrow 124 \rightarrow 183$ Movimento: $|0 - 53| + |183 - 0| = \mathbf{236}$ cil.

4. LOOK Inverte la rotta senza raggiungere gli estremi (0 o 199). Percorso: $53 \rightarrow 37 \rightarrow 14 \rightarrow 65 \rightarrow 67 \rightarrow 98 \rightarrow 122 \rightarrow 124 \rightarrow 183$ Movimento: $|14 - 53| + |183 - 14| = \mathbf{208}$ cil.

5. C-SCAN (Circular SCAN) Percorso: $53 \rightarrow 65 \rightarrow 67 \rightarrow 98 \rightarrow 122 \rightarrow 124 \rightarrow 183 \rightarrow 199 \rightarrow (0) \rightarrow 14 \rightarrow 37$ Movimento (incluso riavvolgimento): $|199 - 53| + |199 - 0| + |37 - 0| = \mathbf{382}$ cil.

6. C-LOOK Percorso: $53 \rightarrow 65 \rightarrow 67 \rightarrow 98 \rightarrow 122 \rightarrow 124 \rightarrow 183 \rightarrow (14) \rightarrow 37$ Movimento: $|183 - 53| + |183 - 14| + |37 - 14| = \mathbf{322}$ cil.

Criteri di Selezione

SSTF è semplice ed efficace, ma soffre di *starvation*. SCAN e C-SCAN sono ideali per carichi pesanti, mentre LOOK e C-LOOK ottimizzano il movimento meccanico evitando corse inutili verso i bordi del disco.

6.3 Calcolo del Tempo di Seek basato su Seek Unitario

Definizione

Se è noto il tempo di seek unitario ($T_{unitario}$), ovvero il tempo necessario per spostare la testina di un singolo cilindro, il tempo di seek totale (T_{seek}) è dato dal prodotto tra tale valore e il numero di cilindri da attraversare ($N_{cilindri}$).

$$T_{seek} = T_{unitario} \cdot N_{cilindri}$$

dove $N_{cilindri} = |C_f - C_i|$.

6.4 Amministrazione e Gestione del Disco

Prima dell'uso, il disco subisce tre fasi:

1. **Formattazione a basso livello:** Creazione dei settori fisici.
2. **Partizionamento:** Divisione in unità logiche indipendenti.
3. **Formattazione logica:** Inizializzazione del File System.

7 Implementazione del File System

7.1 Architettura e Strutture Dati

Il file system è organizzato in strati gerarchici che separano l'interfaccia utente (file system logico) dalla gestione fisica dei dispositivi (controllo I/O). Ogni file è descritto da un **File Control Block (FCB)**, che contiene metadati come permessi, proprietario, date di accesso/scrittura e puntatori ai blocchi dati.

Strutture in Memoria

Per ottimizzare le prestazioni, il sistema utilizza:

- **Tabella dei file aperti (sistema):** Contiene i FCB dei file aperti da tutti i processi.
- **Tabella dei file aperti (processo):** Contiene puntatori alle voci della tabella di sistema e le informazioni sul puntatore al file corrente per il processo.

7.2 Metodi di Allocazione

L'efficienza del file system dipende dalla strategia di allocazione dei blocchi su disco:

1. **Contigua:** Veloce, ma soggetta a frammentazione esterna.
2. **Collegata (Linked):** Liste di blocchi; ottima per file sequenziali, pessima per accesso casuale.
3. **Indicizzata:** Utilizza un blocco indice per raccogliere i puntatori ai dati. Supporta accesso casuale efficiente.

Esercizio 7.2.1

Si consideri un sistema di allocazione indicizzata a 2 livelli con blocchi da 32 kB e puntatori da 8 B. Calcolare la dimensione massima di un file.

Dati

- Dimensione blocco (B): $32 \text{ kB} = 2^{15} \text{ B}$

- Dimensione puntatore (P): $8\text{ B} = 2^3\text{ B}$

Svolgimento

Numero di puntatori per blocco:

$$N_p = \frac{2^{15}}{2^3} = 2^{12}$$

Essendo a 2 livelli (indirezione doppia), il numero totale di blocchi dati è:

$$Tot_{blocchi} = N_p \cdot N_p = 2^{12} \cdot 2^{12} = 2^{24} \text{ blocchi}$$

Dimensione massima file:

$$Dim_{max} = Tot_{blocchi} \cdot B = 2^{24} \cdot 2^{15}\text{ B} = 2^{39}\text{ B}$$

Conversione in GB ($2^{30}\text{ B} = 1\text{ GB}$):

$$Dim_{max} = \frac{2^{39}}{2^{30}}\text{ GB} = \mathbf{512\text{ GB}}$$

7.3 Prestazioni e Recupero

Per migliorare la velocità, il sistema utilizza la **Buffer Cache** (spesso unificata con la cache di pagina della memoria virtuale). Per garantire la coerenza in caso di crash, si utilizzano i **Log-structured File Systems**, che registrano ogni modifica come transazione. In caso di interruzione, il sistema riesegue le transazioni rimaste nel log, garantendo l'integrità dei dati.

Nota sui livelli di indirezione

In un sistema di allocazione indicizzata reale (come in Unix), il calcolo della dimensione massima non si limita al solo livello di indirezione doppia. La capacità totale del file è data dalla somma di:

- **Blocchi diretti:** Puntatori che puntano direttamente ai blocchi dati.
- **Indiretto singolo:** Un blocco che contiene puntatori a blocchi dati.
- **Indiretto doppio:** Un blocco che punta a blocchi di puntatori (come nell'esercizio precedente).
- **Indiretto triplo:** Un blocco che punta a blocchi che puntano a loro volta a blocchi di puntatori.

In presenza di una struttura combinata, la dimensione massima del file si ottiene sommando la capacità di ciascun livello, garantendo così sia un accesso rapido per file piccoli (tramite blocchi diretti) sia una scalabilità elevata per file di grandi dimensioni.